SegAlign: A Scalable GPU-Based Whole Genome Aligner

Goenka S., Turakhia Y., Paten B., Horowitz, M.

Juan José Olivera Loyola

March 20, 2025

Project Charter

Agenda:

- Problem Statement
- Related Work
- Seed-Filter-Extend Pipeline
- Implementation Details
- Experimental Results
- Conclusions

Problem: Whole Genome Alignment (WGA)

Find the complete alignment between a pair of genomes.
(Ex. Human 3Gbp ⇔ Mouse 2.7Gbp or Human 1 ⇔ Human 2).

Alignment Literature

- **BLAST ():** Base tool for DNA and protein sequence alignment
  - Based on Smit&Waterman DP alignment + Seed-Extend Heuristic
  - Cons: Prohibitive expensive memory utilization for WGA.

Problem: Whole Genome Alignment (WGA)

Find the complete alignment between a pair of genomes.
(Ex. Human 3Gbp ⇔ Mouse 2.7Gbp or Human 1 ⇔ Human 2).

Alignment Literature

- **BLAST ():** Base tool for DNA and protein sequence alignment
  - Based on Smit&Waterman DP alignment + Seed-Extend Heuristic
  - Cons: Prohibitive expensive memory utilization for WGA.
- **BLASTZ, LASTZ:** Inspired on BLAST optimized for memory utilization.
  - Using preprocessing of sequences and Gapfree extension filtering
  - Cons: CPU programs

Problem: Whole Genome Alignment (WGA)

Find the complete alignment between a pair of genomes.
(Ex. Human 3Gbp ⇔ Mouse 2.7Gbp or Human 1 ⇔ Human 2).

Alignment Literature

- **BLAST ():** Base tool for DNA and protein sequence alignment
  - Based on Smit&Waterman DP alignment + Seed-Extend Heuristic
  - Cons: Prohibitive expensive memory utilization for WGA.
- **BLASTZ, LASTZ:** Inspired on BLAST optimized for memory utilization.
  - Using preprocessing of sequences and Gapfree extension filtering
  - Cons: CPU programs
- **GPU-BLAST,cuBLASTP,G-BLASTN**
  - Accelerate BLAST throught GPU exploitation
  - Const: Work with proteins. Still huge system requirements for WGA.

Problem: Whole Genome Alignment (WGA)

Find the complete alignment between a pair of genomes.
(Ex. Human 3Gbp ⇔ Mouse 2.7Gbp or Human 1 ⇔ Human 2).

Alignment Literature

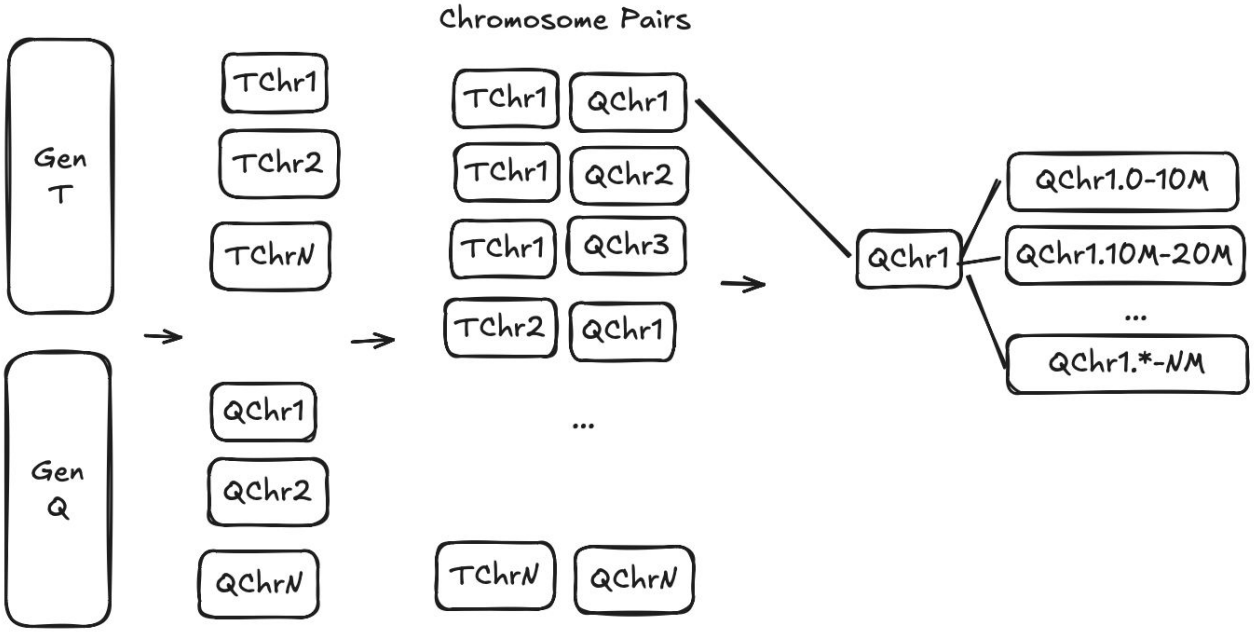
- **BLAST ():** Base tool for DNA and protein sequence alignment
  - Based on Smit&Waterman DP alignment + Seed-Extend Heuristic
  - Cons: Prohibitive expensive memory utilization for WGA.
- **BLASTZ, LASTZ:** Inspired on BLAST optimized for memory utilization.
  - Using preprocessing of sequences and Gapfree extension filtering
  - Cons: CPU programs
- **GPU-BLAST,cuBLASTP,G-BLASTN**
  - Accelerate BLAST throught GPU exploitation
  - Const: Work with proteins. Still huge system requirements for WGA.
- **SegAlign:** Accelerates Seed-Filter stage of LASTZ.
  - CUDA implementation of seed-filter
  - Whole pipeline scalable to multiple nodes

Seed-Filter-Extend Pipeline

1. **Seeding:** Fast Small Localized Positions (Seed Hits)
   a. Based on <u>seed patterns</u> to increase sensitivity
   b. Computed with index and positions tables
   c. Computed for **small sequences** of query genome Q against target genome T.

Seed-Filter-Extend Pipeline

Small Sequences:



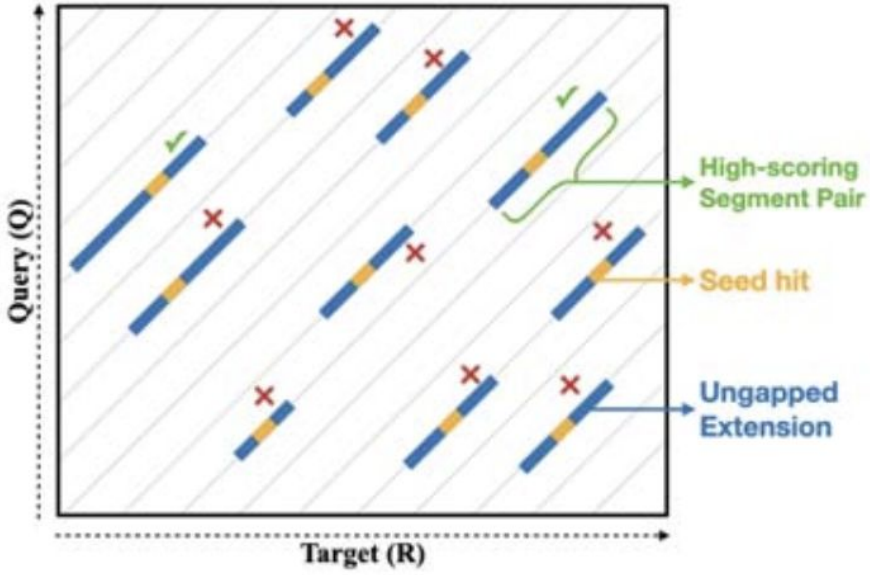
Seed-Filter-Extend Pipeline

1. **Seeding:** Fast Small Localized Positions (Seed Hits)
2. **Filtering**: Gapped-free Extension Filtering
   a. Extend RIGHT and LEFT of seed hit.
   b. Each match-mismatch accumulate to a extension score.
   c. Stop when score < Hx
   d. Filtered extended segments: High-Scoring Segment Pair (HSP)

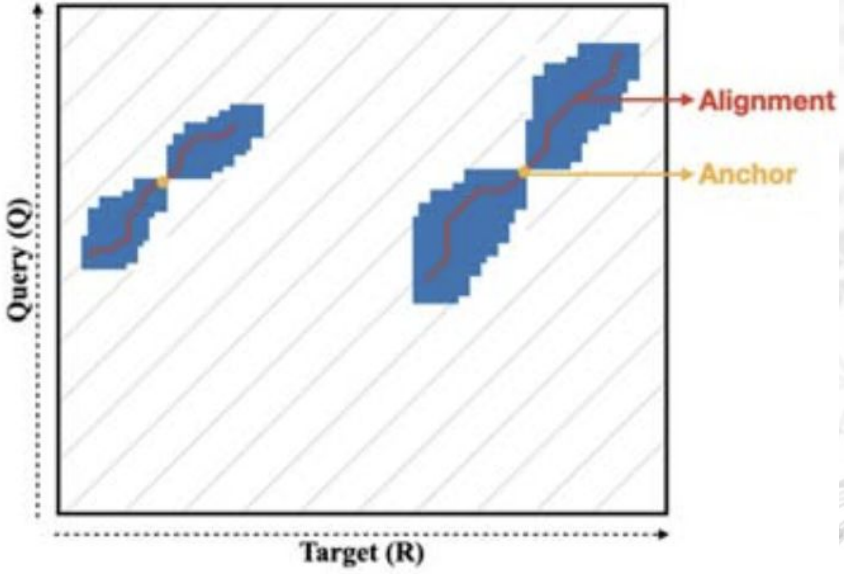
Seed-Filter-Extend Pipeline

1. **Seeding:** Fast Small Localized Positions (Seed Hits)
2. **Filtering**: Gapped-free Extension Filtering
3. **Extend:** DP Alignment Extension
    a. Compute intensive
    b. Extensions with score > He threshold, written to output

Seed-Filter-Extend Pipeline

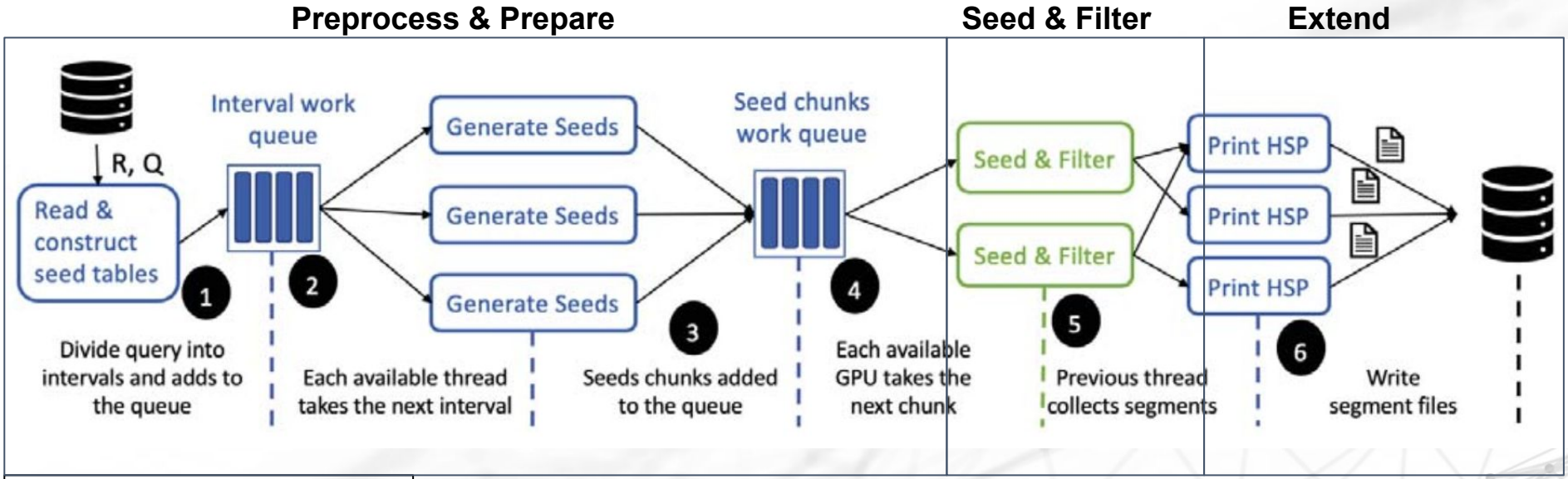


(b)



(c)

SegAlign Program



Blue executed in CPU
Green executed in GPU

SegAlign Program

HPC Principles & Techniques

- GPUs have SMs (Processors) that can thread Single Instruction Multiple Data (SIMD) programs (Kernels).
- To Best Utilize GPU:
  - Kernels should avoid branches to avoid warp stalls
  - Enough thread-blocks provisioned to SM to avoid underutilization
  - Global memory requests should be contiguous
  - On-Chip shared memory should be used when possible
  - To reduced shared memory utilizacion shuffle(__shfl_sync()) for inter-thread communication within a warp

SegAlign Program

- Seeding
  - Preprocessing GPU kernel used to calculate number of seed hits per seed & allocate JUST enough GPU memory for positions.
  - Then a seeding kernel launched
  - 1 seed = 1 GPU thread

SegAlign Program

- Filtering
  - Extending is independent, naive is again 1 seed = 1 thread. But consecutive seeds vary in locations
  - Optimized: 1 seed = 1 warp. Extends by the number of threads, instead per single base.
  - Substitution matrix W per each thread-block

SegAlign Program
TBB-Graph Scheduling

- Work Queues for Intervals, GPU mem copies and HSP are scheduled and parallel processed by TBB threads.
- To arrange dependencies, TBB Flow Graph library was used to define a Compute Graph

SegAlign Program
TBB-Graph Scheduling

Compute Nodes
Edges

```
555         printer_node printer(align_graph, tbb::flow::unlimited, segment_printer_body());
556
557         tbb::flow::function_node<seeder_input, printer_input> seeder(align_graph, tbb::flow::unlimited, seeder_body());
558
559         tbb::flow::make_edge(seeder, printer);
560
561         tbb::flow::join_node<seeder_input> gatekeeper(align_graph);
562
563         tbb::flow::make_edge(gatekeeper, seeder);
564
565         tbb::flow::buffer_node<size_t> ticketer(align_graph);
566
567         // Allocate tickets
568         for (size_t t = 0ull; t < cfg.num_threads; t++)
569             ticketer.try_put(t);
570
571         tbb::flow::make_edge(tbb::flow::output_port<0>(printer), ticketer);
572
573         tbb::flow::make_edge(ticketer, tbb::flow::input_port<1>(gatekeeper));
```

Multiple Nodes Strategy

Technique: MapReduce **=** (1) Parallel Distribute & Compute -> (2) Join

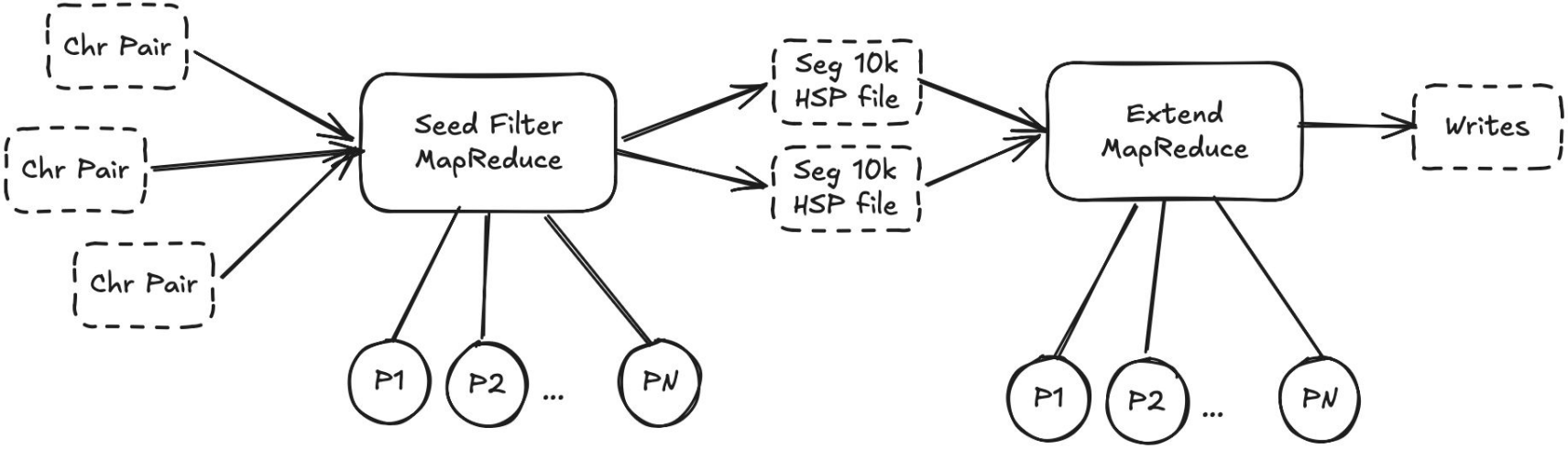
Initial Problem: A lot of variation for different chromosomes in seed-filter and extend stages. (1min to 14mins per pair) => Potential a lot of load imbalance.

Solution: Two MapReduces phases:

- **Seed-Filter:**
  - Each chromosome pair is a task
  - Produces a set of segments files with up to 10k HSPs.
- **Extend:**
  - Each segment file is a task
  - Processed with LASTZ extend.

Better Load Balancing

Multiple Nodes Strategy



Multiple Nodes Strategy

Technique: MapReduce **=** (1) Parallel Distribute & Compute -> (2) Join

Initial Problem: A lot of variation for different chromosomes in seed-filter and extend stages. (1min to 14mins per pair) => Potential a lot of load imbalance.

Solution: Two MapReduces phases:

- **Seed-Filter:**
  - Each chromosome pair is a task
  - Produces a set of segments files with up to 10k HSPs.
- **Extend:**
  - Each segment file is a task
  - Processed with LASTZ extend.

Better Load Balancing

Experimental Setup

AWS Instances and AWS Elastic MapReduce

	Instance Name	# vCPUs	# V100 GPUs	Unitary Cost
LASTZ Baseline	c5.24xlarge	96	0	$4.08/he
Single Node SegAlign	p3.16xlarge	64	8	$24.48/hr
Multi Node SegAlign	p3.2xlarge	8	1	$3.06/hr

Metrics

- Execution Time [s]
- Cost Improvement [$USD/hr]
- Scalability Efficiency [%]

UCSC database

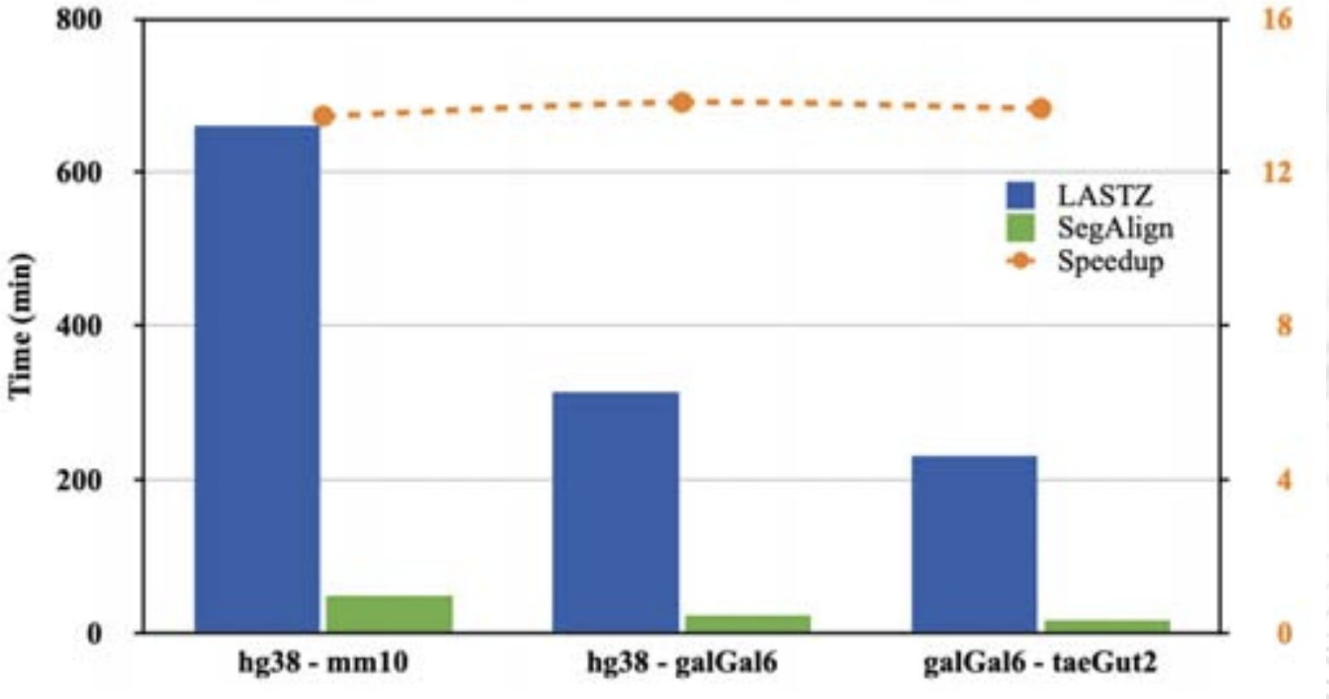
Human-Mouse

Human-Chicken

Chicken-Zebra

Experimental Results - Single Node SegAlign

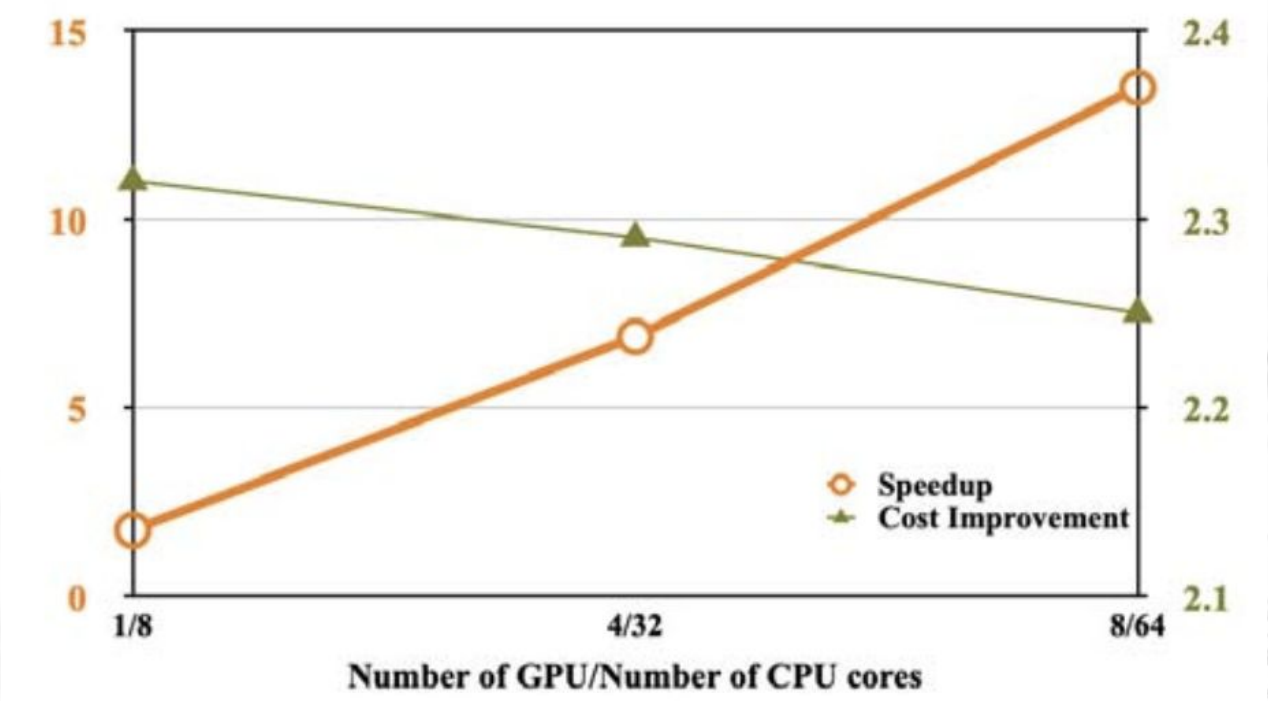
Time of execution and speedup for LASTZ Baseline and Single Node SegAlign



Speedup: 14x

Experimental Results - Single Node SegAlign

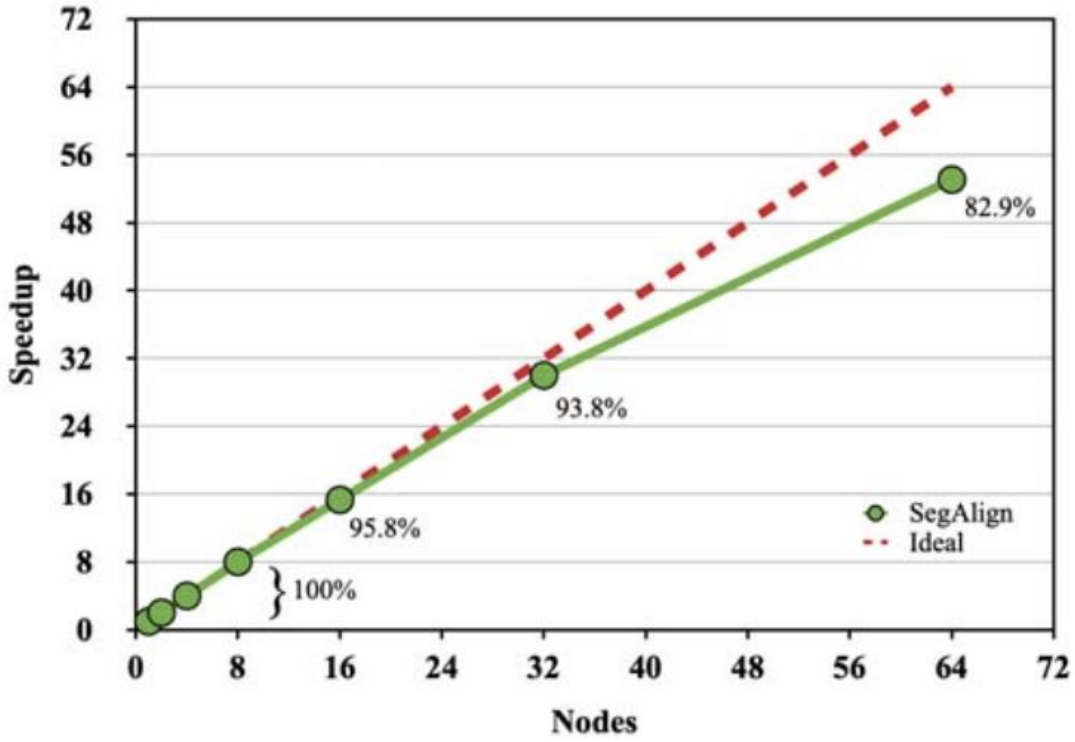
Speedup and Cost Improvement vs Instance Size (GPUs/CPUs)



Cost Improvement: up to 2.25x for 8 GPUs

Experimental Results - MultiNode SegAlign

Strong Scaling: Speedup vs #Nodes



Up to 32 nodes,
parallel efficiency > 90%

64 nodes,
parallel efficiency 82%
speedup: ~50X.

Experimental Results - MultiNode SegAlign

Weak Scaling (Increased Problem Size): Time, Efficiency vs #Nodes

Genome size (Mbp)	Number of Nodes	Time	Efficiency
195	1	44m 25s	100%
390	2	44m 27s	99.9%
780	4	44m 43s	99.3%
1560	8	45m 0s	98.7%
3120	16	45m 20s	98.0%
6240	32	45m 23s	97.9%
12480	64	46m 5s	96.4%

TABLE II: Weak scaling analysis for SegAlign.

Double problem size?
Double compute.

=> Similar Execution Time => (~96%) Parallel Efficiency

Conclusion

- The Whole Genome Alignment (WGA) was accelerated by the SegAlign program achieving in an WGA experiment:
  - ~14x speedup
  - ~2.25x cost improvement
  - good strong scaling 86% with 64 nodes
  - very good weak scaling 96% with 64 nodes

  compared against parallelized state-of-the-art LASTZ Tool.
- It employs GPU CUDA parallelization for performing the seed-filter-extend
- High memory and communication aware optimizations
- Multithreaded CPU to schedule operations according a compute graph with TBB.
- It is integrated with ApacheSpark to embarrassingly distribute the workload across different nodes and reduce to a global alignment mapping.
- It serves as a drop-off replacement for LASTZ.

Use of AI

ChatGPT to

- Help understand paper and WGA literature
- Solve issues compiling
- Understand source code

References

- **Goenka S., Turakhia, Y., Paten, B., Horowitz, M. (2020). “SegAlign: A Scalable GPU-Based Whole Genome Aligner”.**
- Harris R.S. (2007). “Improved pairwise alignment of genomic DNA”.
- Smith, T., Waterman, M. (1981).“Comparison of Biosequences”.
- Ahmid, A., et al. (2011). “Hobbes: optimized gram-based methods for efficient read alignment”.